

Cassandra Web Sanitizer - Report Programmazione di Sistema

Introduzione

Cassandra Web Sanitizer è un tool di sicurezza difensivo scritto in Rust. Funge da livello di sanitizzazione intermedio intercettando pagine web non attendibili tramite link, strutture di file locali e risorse scaricate dalla rete, per poi neutralizzare o riscrivere potenziali elementi pericolosi prima che possano causare danni all'utente.

Cassandra è progettato come un sistema modulare composto da due componenti:

1. Una libreria riutilizzabile (`sanitizer_engine`) che implementa il motore di scansione, i crawler di rete ricorsivi, il rilevamento dei file tramite magic-number (sniffing), i riscrittori di flusso (stream rewriter), i parser di documenti...
2. Un'applicazione da riga di comando (`cli_application`) che permette di utilizzare la libreria di sanitizzazione attraverso un'interfaccia grafica.

La progettazione si affida interamente a **safe Rust**, impiegando una tecnica di **token streaming zero-copy** (utilizzando `lol_html`) al fine di evitare il sovraccarico prestazionale e le vulnerabilità di sicurezza legate alla materializzazione di interi alberi DOM in memoria. Per valutarne l'efficacia, Cassandra è stato testato rispetto a una suite di test contenente sia pagine benigne che risorse pericolose (XSS, XML entity bomb, elementi PDF attivi, confusione omografa IDN, SSRF e MIME confusion). La valutazione sperimentale mostra che Cassandra ottiene un **tasso di rilevamento del 100%** sul corpus malevolo, mantenendo un footprint di memoria di picco estremamente ridotto pari a **49,80 MB** e dimostrando una buona scalabilità su CPU parallele, limitata tuttavia dalla fase sequenziale di scrittura del log su disco.

1. Analisi dei Requisiti

1.1 Requisiti Funzionali

L'architettura funzionale di Cassandra è guidata dalla necessità di difendere il sistema dell'utente. Il sistema neutralizza sei principali minacce:

1. Sanitizzazione dell'HTML:

- **Gestori di Eventi Inline:** Rimozione degli attributi di evento (es. `onclick` , `onerror` , `onload` , `onmouseover` , `onfocus`) da tutti i tag HTML.
- **Blocchi Script:** Neutralizzazione o rimozione dei tag `<script>` a meno che il loro hash di contenuto o il dominio sorgente non corrispondano esplicitamente a un elemento in whitelist.
- **Protocolli Pericolosi:** Intercettazione e sanitizzazione di URI `javascript:` e `data:` presenti negli attributi `href` e `src` .

- **Restrizioni sulle Origini:** Verifica dei tag `<iframe src="...">` e `<object data="...">`, rimuovendoli o bloccandoli se puntano a domini esterni all'elenco delle origini approvate.
- **Reindirizzamenti Meta:** Rimozione completa dei tag `<meta http-equiv="refresh">` per bloccare attacchi di auto-redirezione lato client.
- **Estrazione Estesa dei Tag:** Analisi e sanitizzazione dei collegamenti e delle risorse all'interno di elementi nidificati, inclusi `<form>`, `<audio>`, `<video>`, `<embed>`, `<track>`, `<area>` e `<input>`.

2. MIME Confusion e Content Sniffing:

- Convalida proattiva delle risorse ignorando l'intestazione `Content-Type` fornita dal server HTTP remoto. Il motore esamina i primi byte (magic numbers) dei file per determinare il loro tipo effettivo (HTML, CSS, JS, JPEG, PNG, PDF) e rifiuta i file con intestazioni non corrispondenti.

3. Attacchi Omografi Unicode:

- Rilevamento e prevenzione del domain spoofing mediante la verifica degli URL estratti. Il componente del dominio viene analizzato, normalizzato e controllato alla ricerca di pattern di omografi IDN (Internationalized Domain Name), neutralizzando i collegamenti che usano glifi visivamente simili.

4. Scansione dei Documenti con Contenuto Attivo:

- Ispezione dei documenti PDF per individuare contenuti attivi eseguibili integrati. Esegue una scansione della struttura binaria del PDF alla ricerca dei dizionari `/JavaScript`, `/JS` o `/OpenAction` che attivano comportamenti dinamici all'interno dei lettori PDF, neutralizzando il file in caso di rilevamento.

5. Protezione da attacchi Denial of Service (DoS):

- **Compression Bomb:** Limitazione dei rapporti di decompressione e della dimensione massima dei byte decompressi per le risorse web compresse (es. codifiche `gzip` o `deflate`).
- **Espansione di Entità XML/HTML:** Blocco delle dichiarazioni di entità XML personalizzate (es. i riferimenti ricorsivi di tipo `<!ENTITY lol "lol">`) per difendersi dagli attacchi DoS di espansione.
- **Dimensioni delle Immagini ed Esaurimento Risorse:** Filtro su asset di dimensioni eccessive o risorse potenzialmente infinite.

6. Server-Side Request Forgery (SSRF) e DNS Rebinding:

- Durante il recupero delle risorse remote, blocco dell'accesso a indirizzi di loopback (`127.0.0.0/8`, `::1`), subnet private (`10.0.0.0/8`, `192.168.0.0/16`, `172.16.0.0/12`), spazi link-local, indirizzi multicast e intervalli CGNAT (Carrier-Grade NAT) per evitare che Cassandra venga usato per effettuare scansioni di reti interne. Inoltre, l'integrazione di un resolver DNS personalizzato direttamente nel livello di connessione del client HTTP (`SsrfSafeDnsResolver`) garantisce la verifica atomica degli IP al momento dell'apertura del socket TCP, prevenendo gli attacchi di DNS Rebinding.

Reportistica Strutturata e Configurazione CLI:

Cassandra scrive un file consolidato `cassandra.log` e un singolo report strutturato `report.json` in cui viene verificata ogni azione (regole attivate, offset dei byte, valori originali e modifiche) per ciascun input.

Supporta inoltre configurazioni della CLI tramite file di policy TOML personalizzati, esecuzione batch, impostazione del numero di thread worker e codice di uscita non nullo se le regole di blocco della policy risultano in un rifiuto del contenuto.

1.2 Requisiti Non Funzionali

- **Sicurezza della Memoria:** Assenza totale di vulnerabilità legate alla gestione della memoria (es. buffer overflow, use-after-free) garantendo che i livelli di parsing e riscrittura siano scritti in safe Rust.
 - **Footprint di Memoria Minimo:** Elaborazione di file di grandi dimensioni e compiti batch senza caricare i documenti interi in memoria, implementando tokenizer di flusso per limitare l'uso della RAM.
 - **Concorrenza e Scalabilità dei Thread:** Distribuzione dei compiti di elaborazione sui core della CPU disponibili, utilizzando scheduler asincroni per gestire in parallelo letture del disco e I/O di rete.
 - **Budget di Tempo e Risorse:** Applicazione di limiti superiori rigidi sulla profondità di recupero delle risorse (`max_depth`), sul numero totale di richieste HTTP (`max_requests`) e sui byte totali scaricati (`max_bytes`) per garantire che l'elaborazione termini sempre in modo sicuro anche sotto attacco.
-

2. Decisioni Architettureali

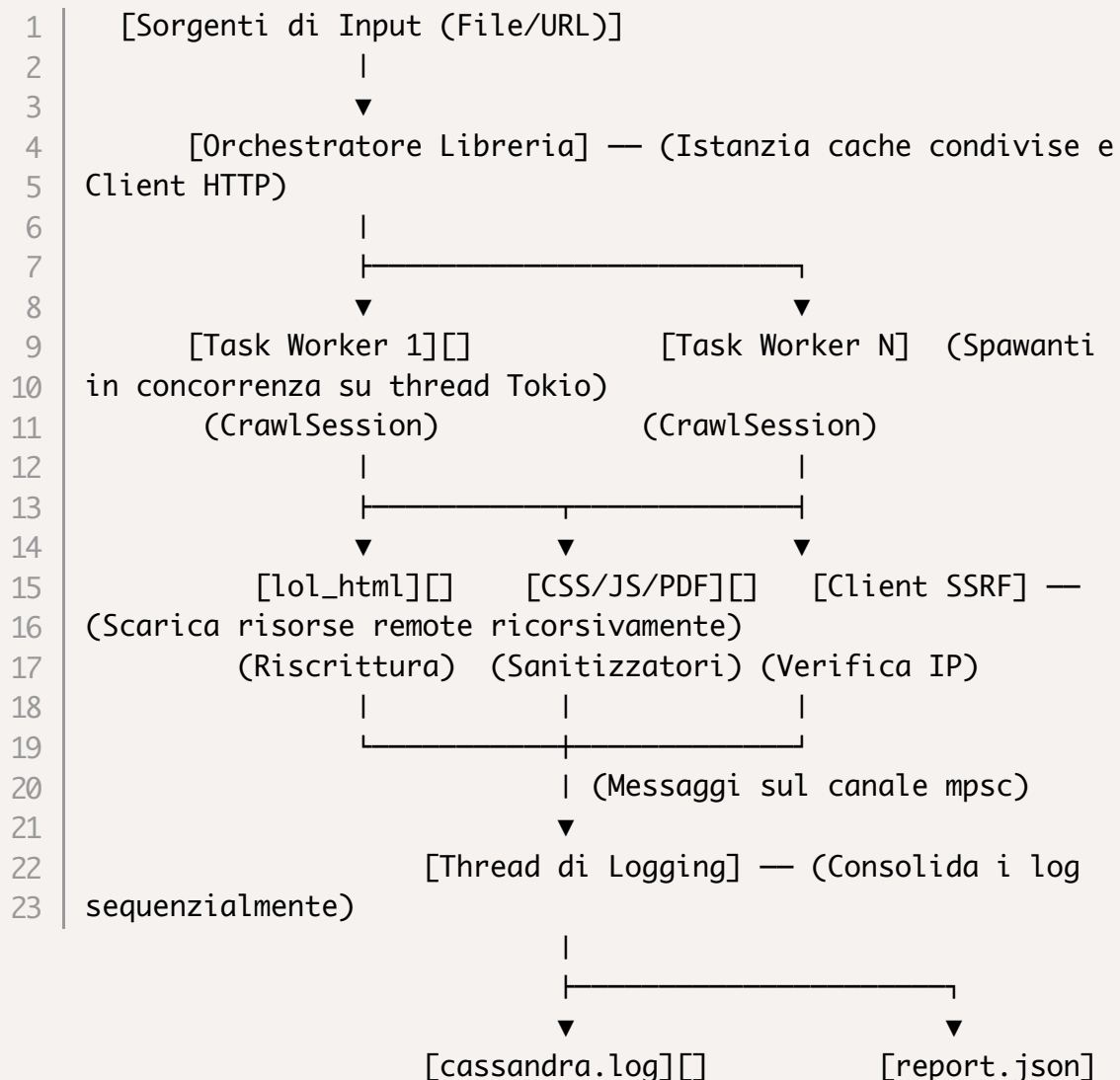
Cassandra è organizzato come un workspace Cargo multi-crate per separare nettamente la logica del motore di sanitizzazione dall'interfaccia utente della CLI.

```

1 | cassandra/ (Workspace Root)
2 |   └─ Cargo.toml
3 |   └─ policies/
4 |       └─ default.toml      <-- Policy Predefinita
5 |   └─ sanitizer_engine/     <-- Crate Libreria Core
6 |   (cassandra)
7 |       └─ Cargo.toml
8 |       └─ src/
9 |           └─ lib.rs        <-- Orchestratore della Libreria
10 |           └─ crawl_session.rs <-- Crawler
11 |           └─ http_client.rs <-- Client HTTP Safe Anti-SSRF
12 |           └─ html.rs       <-- Riscrittore HTML in
13 |   streaming (lol_html)
14 |       └─ log.rs           <-- Thread di Logging su Canale
15 |   MPSC
16 |       └─ errors.rs        <-- Errori Personalizzati
17 |       └─ policy.rs        <-- Definizione Policy TOML
18 |       └─ rules.rs        <-- Definizioni delle regole
19 |       └─ url.rs          <-- Verifica URL e IDN
20 |       └─ resources/      <-- Sanitizzatori Specifici per
21 |   Risorse
22 |       └─ mod.rs          <-- Inizializzatore Modulo e
23 |   Gestore Nomi File
24 |       └─ css.rs          <-- Sanitizzatore CSS
25 |       └─ entities.rs     <-- Scanner Entità XML (Entity
26 |   Bomb)
27 |       └─ images.rs       <-- Rimozione Metadati e EXIF
28 |   Immagini
29 |       └─ javascript.rs   <-- Scanner Parole Chiave JS
30 |       └─ mime.rs         <-- Rilevatore Magic Number
   |   (Sniffer)
   |       └─ pdf.rs        <-- Scanner Contenuti Attivi PDF
   |   └─ cli_application/  <-- Crate Frontend CLI
   |   (cli_application)
   |       └─ Cargo.toml
   |       └─ src/
   |           └─ main.rs    <-- Gestore ed Esecutore dei
   |   Comandi CLI
   |       └─ bin/
   |           └─ evaluation_runner.rs <-- Runner della
   |   Valutazione Sperimentale

```

2.1 Pipeline di Elaborazione Dati



- **L'Orchestrator** (`lib.rs`): Inizializza una mappa globale thread-safe delle URL visitate (`url_map` protetta da `Arc<Mutex<HashMap<Url, usize>>>`) e istanzia il client HTTP safe. Itera sulle sorgenti di input, costruisce le istanze di `CrawlSession` e le pianifica sull'esecutore asincrono di Tokio.
- **La Crawl Session** (`crawl_session.rs`): Gestisce l'elaborazione dei singoli file e il crawling delle URL. Tiene traccia delle limitazioni delle risorse (`total_requests` , `total_bytes`) per sessione utilizzando contatori locali protetti da mutex e genera asincronamente nuove richieste per le risorse identificate (es. fogli di stile, script) fino alla profondità massima `max_depth` .
- **Il Client HTTP Safe** (`http_client.rs`): Intercetta le richieste in uscita e applica un resolver DNS personalizzato (`SsrfSafeDnsResolver`) integrato direttamente in `request` , eseguendo la validazione IP al momento dell'apertura del socket TCP. Questo approccio previene attacchi TOCTOU e DNS Rebinding, filtrando indirizzi IP privati, loopback e CGNAT per ogni connessione e

reindirizzamento. Limita la comunicazione alle sole connessioni HTTPS e analizza i magic numbers dei corpi delle risposte.

- **Parser HTML in Flusso** (`html.rs`): Integra `lol_html` . Legge i file HTML in piccoli chunk, applicando riscrittori basati su selettori CSS per intercettare elementi critici (es. `<script>` , `<iframe>` , `<a>`) e modificarne le proprietà al volo senza allocare memoria superflua.
- **Il Logger Consolidato** (`log.rs`): I worker inviano log di avanzamento ed errori di regola (`RuleError`) tramite un canale `mpsc` . Un thread in background dedicato legge dal canale, stampando il progresso sulla riga di comando. Genera poi il file `cassandra.log` e produce il report strutturato finale `report.json` .

NOTA Impatto sul consumo di RAM

I dati accumulati in memoria per il log e il report sono strutturalmente molto leggeri:

- Una riga di log media occupa circa un centinaio di byte.
- Anche nel nostro scenario più pesante (carico grande da **7000 file** con circa 15.000 righe di log e report), l'occupazione totale in RAM delle stringhe e dei vettori accumulati è di circa **1.5 - 2.5 MB**.
- Poiché il consumo di RAM di picco misurato è di circa **50 MB** (occupati principalmente da runtime Tokio, stack dei thread, buffer di connessione socket e strutture dati globali), liberare 2 MB di RAM non darebbe alcun beneficio tangibile al sistema.

2.2 API Pubbliche del Crate

Il crate di libreria (`sanitizer_engine`) espone un'interfaccia pubblica che permette l'integrazione di Cassandra come dipendenza all'interno di altre applicazioni Rust. Le API principali fornite sono:

- **InputSource** : Enum che rappresenta la sorgente da scansionare, supportando file locali o URL remoti:

```
rust
```

```
pub enum InputSource {
```

```
    File(PathBuf),
```

```
    Url(Url),
```

```
}
```

- **library(...)** : La funzione di ingresso principale per l'esecuzione della pipeline di sanitizzazione. Accetta un riferimento al runtime di Tokio, una lista di sorgenti, la policy di sicurezza, il percorso di output e il canale MPSC per la trasmissione asincrona dei messaggi di log:

```
rust
```

```
pub fn library(
```

```

runtime: &Runtime,

sources: Vec<InputSource>,

policy: Arc<Policy>,

output_dir: Arc<PathBuf>,

tx: Sender<LoggerMessage>,

) -> Result<(), SanitizerError>;

```

- **logging_thread(...)** : Funzione pubblica esposta dal modulo `log` per avviare il ciclo di consumo sincrono dei messaggi dal canale MPSC, incaricata di generare i file consolidati `cassandra.log` e `report.json` :

```

rust

pub fn logging_thread(

    output: &Path,

    console_level: LogLevel,

    file_level: LogLevel,

    sources: &[InputSource],

    max_subresources: usize,

    channel: Receiver<LoggerMessage>,

) -> (bool, f64, f64);

```

- **Policy** : La struttura di configurazione che definisce le regole di sanitizzazione (esprese in TOML), suddivisa in sotto-policy: `HtmlPolicy` , `UrlsPolicy` , `ResourcesPolicy` e `ConnectionsPolicy` .
- **RuleWithValue<T>** e **ReplaceRule<T>** : Esposte nel modulo `rules` , permettono di associare livelli di log personalizzati alle limitazioni numeriche (es. `MaxBytes` o `MaxSubresources`) e alle azioni di sostituzione.
- **Sanitizzatori Specifici (`resources/`)**: La libreria espone funzioni di utilità riutilizzabili per manipolazioni mirate, tra cui `strip_jpeg_metadata` / `strip_png_metadata` per la rimozione EXIF, ed `EntityScanner` per il rilevamento incrementale di entità XML sospette.

2.3 Patch alla libreria url

Per quanto riguarda la libreria url la versione originale è stata modificata in modo che fosse tenuta traccia dell'url originale in caso di sostituzione di caratteri IDN. In questo modo, anche dopo la sostituzione, è possibile scrivere a log il valore originario.

3. Aspetti di System Programming in Rust

3.1 Modello di Concorrenza e Primitive di Sincronizzazione

Cassandra si affida al **runtime multi-thread di Tokio** per eseguire in parallelo le operazioni CPU-bound ed I/O-bound. Per coordinare i dati tra i thread in totale sicurezza, il motore utilizza le seguenti primitive di concorrenza Rust:

- `Arc<Mutex<HashMap<Url, usize>>>` : Utilizzato per la cache globale delle URL visitate. I worker controllano questa mappa per evitare download duplicati. Poiché le ricerche sono immediate, il lock viene mantenuto per pochissimo tempo, riducendo al minimo la contesa. Si è preferito `parking_lot::Mutex` rispetto a `std::sync::Mutex` per via del suo comportamento di spin-lock più veloce sui percorsi ad alta contesa.
- `Arc<Policy>` : Le policy dichiarative sono in sola lettura durante l'esecuzione. Avvolgere la policy in un `Arc` consente ai worker di fare riferimento alle regole contemporaneamente senza il sovraccarico di un lock.
- `std::sync::mpsc::channel` : Si è scelto un canale asincrono di passaggio messaggi per separare l'elaborazione dalla scrittura fisica su disco. Se i worker scrivessero contemporaneamente sul file di log, rimarrebbero bloccati a causa dei lock del filesystem. Invece, i worker inviano i log al canale e un unico thread in background gestisce la scrittura in modo sequenziale.
- **Tokio Runtime Handles**: I thread worker ricevono una copia di `tokio::runtime::Handle` per avviare ricorsivamente compiti di crawling di sottorisorse senza dover passare l'intero oggetto runtime, mantenendo l'orchestrazione dei task leggera.

```
1 // crawl_session.rs - Spawning asincrono ricorsivo delle
2 sottorisorse
3 let clone = Arc::clone(self);
4 self.rt_handle.spawn(async move {
5     if let Err(e) = clone
6         .crawl_subresource(url, local_name, depth, &logger)
7         .await
8     {
9         logger.error(e);
10    }
11 });
```

3.2 Lifetime, Ownership e Parsing Zero-Copy

Gli strumenti di sicurezza devono elaborare flussi di byte non attendibili senza introdurre colli di bottiglia prestazionali. Cassandra ottiene un throughput elevato combinando l'approccio zero-copy con copie di memoria altamente ottimizzate:

- **Tokenizzazione in Flusso:** Utilizzando `lol_html`, il motore elabora i file HTML all'interno di buffer di finestra di 8KB. Analizza, trasforma e scrive i byte direttamente nel flusso del file di output senza generare un DOM in memoria. L'occupazione di memoria rimane costante indipendentemente dalla dimensione del file.
- **Zero-Copy vs Copie Ottimizzate:** Il parsing HTML e la validazione dei percorsi stringa sono strettamente *zero-copy*. Le sottorisorse binarie o complesse (immagini JPEG/PNG, fogli CSS e PDF) vengono invece bufferizzate interamente in memoria per consentire l'accesso casuale e l'analisi strutturale (es. verifica dei chunk PNG o dei dizionari PDF). La riscrittura di queste risorse non è strettamente zero-copy poiché alloca un nuovo buffer per i byte purificati (es. escludendo i segmenti EXIF); tuttavia, l'algoritmo evita qualsiasi deserializzazione intermedia pesante (come la decodifica dei pixel o la materializzazione di un albero sintattico PDF), limitandosi a copiare selettivamente e linearmente i segmenti sicuri del buffer originale.
- **Borrowing vs Allocating:** Dove possibile, le strutture dati prendono in prestito slice di stringhe (`&str`) dai buffer di origine anziché allocare stringhe proprietarie (`String`) sull'heap. I lifetime (es. `'a`) vincolano le regole alla durata del buffer di input, garantendo che la memoria venga liberata non appena il parser avanza.
- **Copy-on-Write (`Cow<'a, str>`):** Utilizziamo `Cow` durante la corrispondenza dei pattern. Se una stringa è pulita, restituiamo un riferimento preso in prestito (`Cow::Borrowed(&str)`); se la stringa richiede una normalizzazione (es. rimozione di spazi bianchi o normalizzazione Unicode), allochiamo memoria e restituiamo una copia proprietaria (`Cow::Owned(String)`), risparmiando cicli CPU sui testi sicuri.

3.3 Gestione degli Errori Type-Safe e Mappatura delle Azioni di Policy

La gestione degli errori in Cassandra è progettata per differenziare nettamente i guasti di sistema (es. fallimento della scrittura di un file) dai blocchi imposti dalle policy di sicurezza (es. blocco di uno script).

- **Enum Type-Safe:** Le strutture degli errori personalizzate sono definite utilizzando la libreria `thiserror` :

```

1  #[derive(Debug, Clone, Serialize, Deserialize)]
2  #[serde(tag = "type", rename_all = "snake_case")]
3  pub enum RuleError {
4      MimeMismatch { expected: Option<String>, actual:
5  Option<String> },
6      BlockedMetaRefresh { original: String },
7      ActiveContent { file: String, content: String },
8      Replace {
9          inner: RuleReplaceError,
10         replacement: Option<String>,
11         offset: Range<usize>,
12     },
13     // ...
14 }

```

- **Mappatura delle Azioni di Policy:**

Le regole hanno un livello di gravità associato:

- ignore
- trace
- info
- warn
- error

Quando il programma identifica la violazione di una regola:

Se la violazione è qualsiasi cosa che non sia `ignore`, la violazione è scritta sul log.

Altrimenti se il livello è un `error`, il programma smette di processare la risorsa.

Alcune regole possono essere configurate con un messaggio di replacement customizzabile.

```

1 // log.rs - Convalida degli errori e gestione delle azioni di
2 policy
3 pub fn handle<T: Into<SanitizerMessage>>(self, logger: &impl
4 Log, message: T) -> Result<(), T> {
5     if self == LogLevel::Error {
6         Err(message)
7     } else {
8         logger.log(self, message);
9         Ok(())
10    }
11 }

```

3.4 Analisi del Codice Unsafe

Cassandra è scritto interamente in safe Rust per ridurre al minimo i bug di memoria.

4. Valutazione Sperimentale

La valutazione sperimentale è stata condotta su un sistema macOS eseguendo `cli_application/src/bin/evaluation_runner.rs` ed elaborando i grafici tramite `plot_results.py`.

4.1 Correttezza

La correttezza del motore di sanitizzazione è stata valutata rispetto a una serie di dati reali rappresentanti una vasta gamma di minacce: cross-site scripting (XSS), XML bomb, IDN, richieste di rete SSRF, bypass di fogli di stile CSS ed elementi binari attivi (PDF con codice JavaScript).

Riepilogo delle Metriche

- **True Positives (TP):** 11
- **True Negatives (TN):** 2
- **False Positives (FP):** 1
- **False Negatives (FN):** 0
- **Tasso di Rilevamento Globale (Sensibilità):** 100,00%
- **Tasso di Falsi Positivi:** 33,33%

Risultati Dettagliati

File	Verdetto Atteso	Verdetto Effettivo	Regole Attese	Regole Effettive	Stato
malicious/untrusted_origins.html	malicious	malicious	dangerous_domain, dangerous_origins	dangerous_domain, dangerous_origins	MATCH (TP)
malicious/unicode_confusion.html	malicious	malicious	idn	idn	MATCH (TP)
malicious/xml_bomb.html	malicious	malicious	xml_entity_declaration	xml_entity_declaration	MATCH (TP)
malicious/ssrf_attempt.html	malicious	malicious	dangerous_scripts	dangerous_scripts	MATCH (TP)
malicious/malicious_urls.html	malicious	malicious	dangerous_domain, idn	dangerous_domain, idn	MATCH (TP)
benign/clean_doc.pdf	benign	benign	none	none	MATCH (TN)
malicious/idn_only.html	malicious	malicious	idn	idn	MATCH (TP)
benign/safe.html	benign	benign	none	none	MATCH (TN)
malicious/pdf_js_bomb.pdf	malicious	malicious	active_content	active_content	MATCH (TP)
malicious/dangerous_script.js	malicious	malicious	dangerous_js	dangerous_js	MATCH (TP)

Analisi dei Falsi Positivi: Il file `benign/crawler_test.html` ha attivato la regola `dangerous_scripts` in quanto conteneva uno script inline non presente nella whitelist predefinita. Poiché il parser è configurato in modalità strettamente difensiva, qualsiasi blocco di script non espressamente approvato viene

categorizzato come non sicuro e sanitizzato, producendo un falso positivo sui siti che usano codice inline senza aver registrato gli opportuni hash.

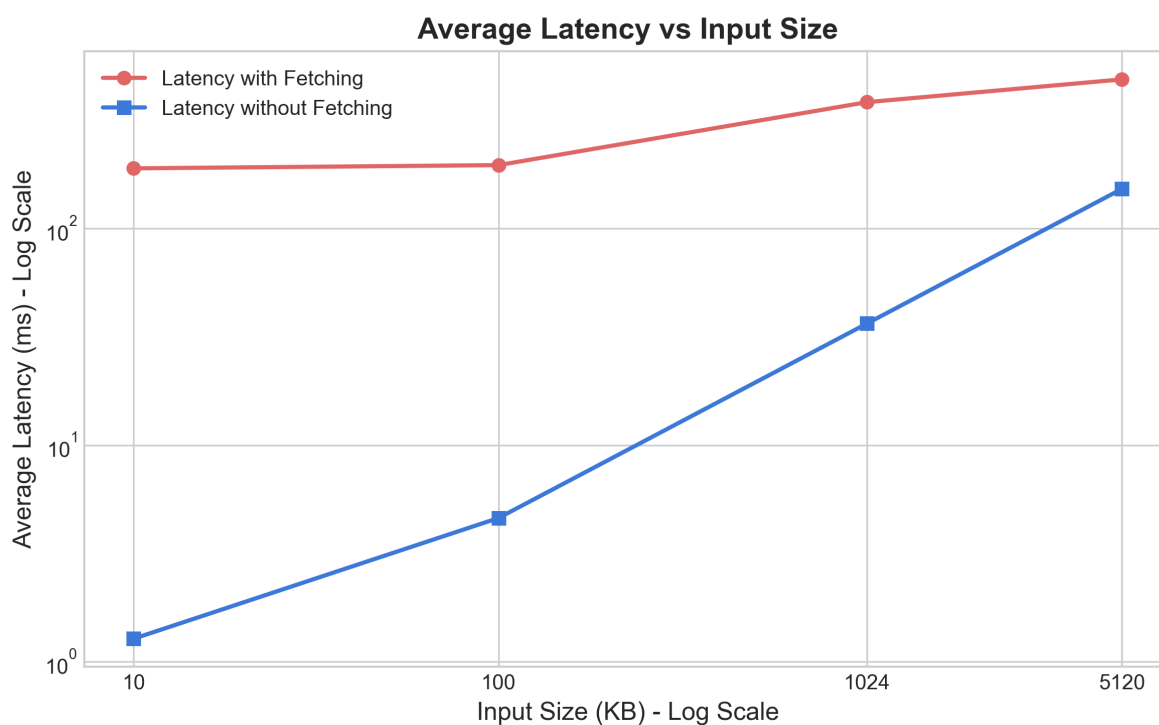
4.2 Performance (Latenza vs Dimensione dell'Input)

Il throughput e la latenza sono stati misurati utilizzando file HTML sicuri con dimensioni variabili da 10KB a 5MB, abilitando e disabilitando il caricamento delle risorse remote collegate.

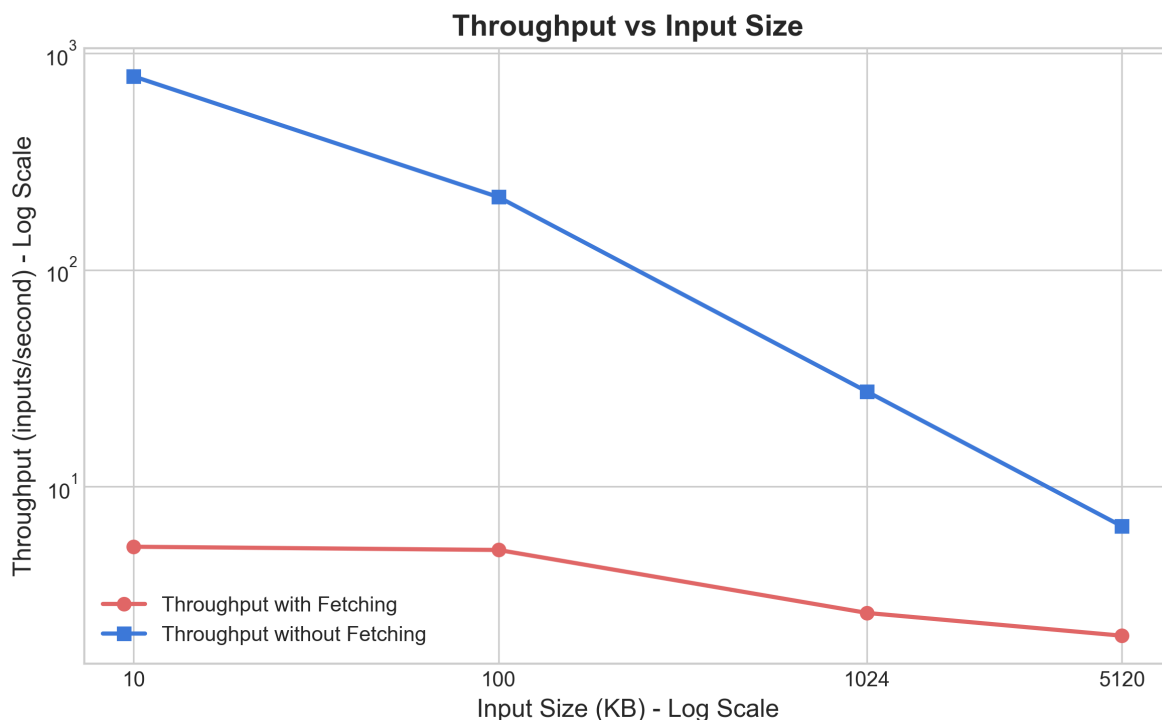
Le tabelle seguenti misurano la latenza per-input in *ms*, e il throughput in *inputs per second*.

Dimens ione	Latenza (No Fetch)	Throughput (No Fetch)	Latenza (Con Fetch)	Throughput (Con Fetch)
10 KB	1.28 ms	783.22 ips	189.64 ms	5.27 ips
100 KB	4.61 ms	217.12 ips	196.23 ms	5.10 ips
1 MB	36.44 ms	27.44 ips	383.32 ms	2.61 ips
5 MB	152.54 ms	6.56 ips	488.34 ms	2.05 ips

Latenza rispetto alla dimensione dell'input



Throughput rispetto alla dimensione dell'input



Osservazioni:

- **Andamento Asintotico Lineare ($O(N)$) e Costi Fissi:** Quando opera esclusivamente come parser locale (senza scaricare le risorse remote), la latenza è dominata dalla scansione dei token HTML. Per file piccoli (da 10KB a 100KB), la latenza passa da **1,28 ms** a **4,61 ms** a causa dell'overhead fisso di setup. Su file voluminosi l'overhead viene ammortizzato: passando da 1MB (**36,44 ms**) a 5MB (**152,54 ms**), la latenza scala in modo proporzionale alla dimensione del payload, confermando l'andamento lineare $O(N)$ della libreria `lol_html`.
- **Mitigazione della Concorrenza sul Subfetching:** Con il download delle sottorisorse abilitato, l'esecuzione sfrutta il client HTTP asincrono in Tokio. La latenza passa da **189,64 ms** (10KB) a **488,34 ms** (5MB), con una crescita notevolmente sub-lineare grazie alla gestione parallela non-bloccante I/O delle connessioni HTTP remote.

4.3 Scalability ed Efficienza della Pipeline Parallela

La scalabilità del sistema è stata valutata confrontando **tre differenti tipologie di carico** al variare del numero di thread worker del runtime Tokio (1, 2, 4, 8 e 16):

1. **Carico Piccolo (140 file piccoli):** Batch rapido compost da 140 file aventi dimensione media di **~305 Byte** (range da 83 Byte a 562 Byte per file), per un volume complessivo di **~41,7 KB (0,04 MB)**. Utile per valutare l'overhead di scheduling.
2. **Carico Grande (7000 file piccoli):** Batch ad alto numero di file (7000 file piccoli, dimensione media **~305 Byte** per file) per un volume complessivo di **~2,04 MB (2.134.500 Byte)**. Utile per saturare la pipeline concorrente con un numero

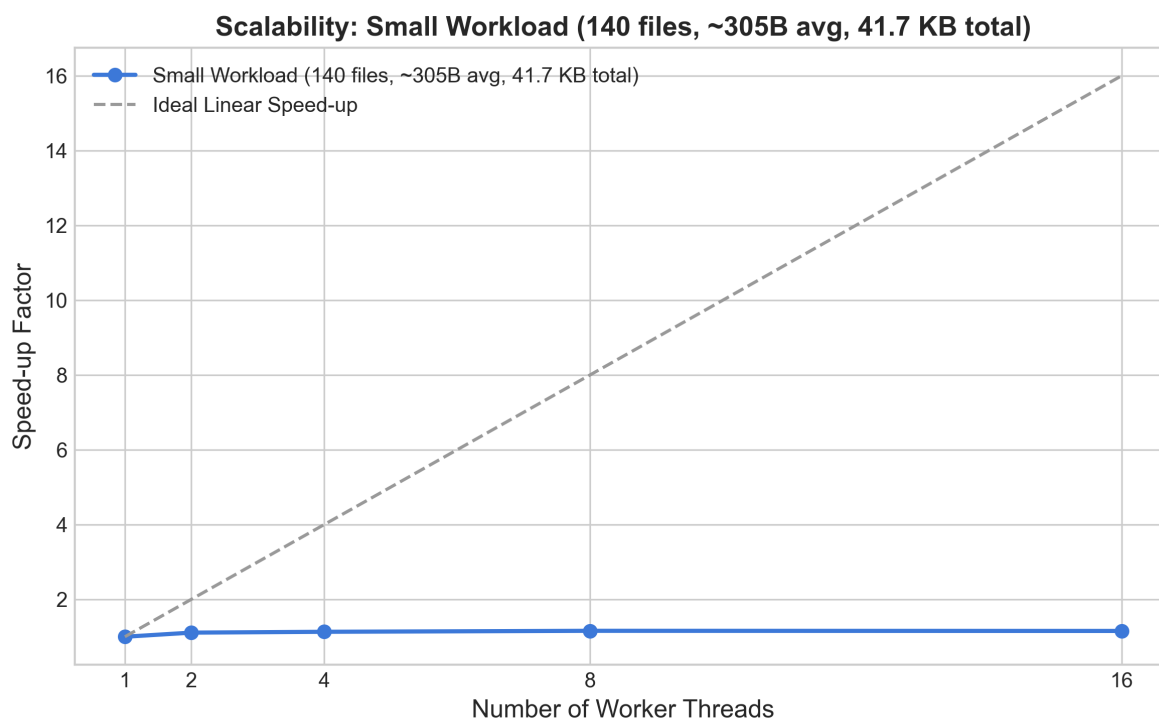
elevato di sorgenti.

3. **Carico Pochi File Grandi (20 file da 5MB):** Batch ad alto volume di payload per singolo file (20 file HTML da 5,24 MB ciascuno) per un volume complessivo di **~100 MB** (104.857.360 Byte), per valutare la scalabilità su elaborazioni CPU-bound pesanti.

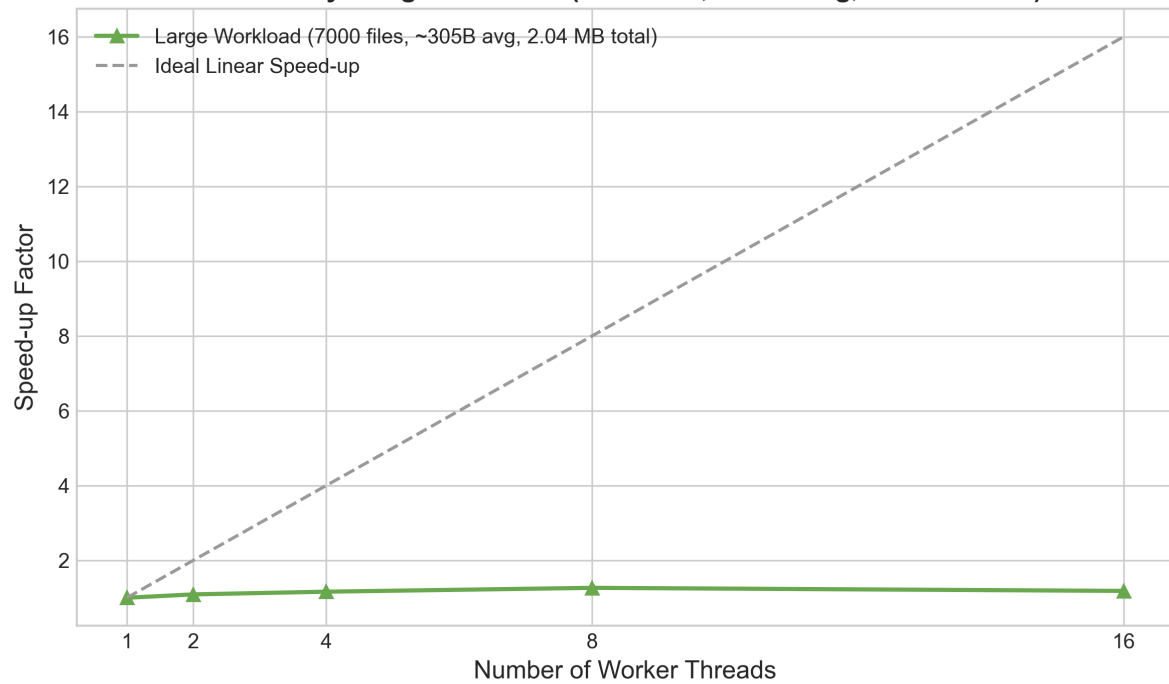
Prestazioni di Scalabilità (Durate di Esecuzione e Speedup)

Numer o Threa d	Durat a Piccol o	Speedu p Piccolo	Durat a Grand e	Speedu p Grande	Durata Pochi File Grandi	Speedup Pochi File Grandi
1	0.063 s	1.00x	2.730 s	1.00x	3.738 s	1.00x
2	0.057 s	1.11x	2.505 s	1.09x	3.219 s	1.16x
4	0.055 s	1.13x	2.349 s	1.16x	2.277 s	1.64x
8	0.054 s	1.16x	2.158 s	1.27x	2.715 s	1.38x
16	0.054 s	1.16x	2.308 s	1.18x	3.265 s	1.15x

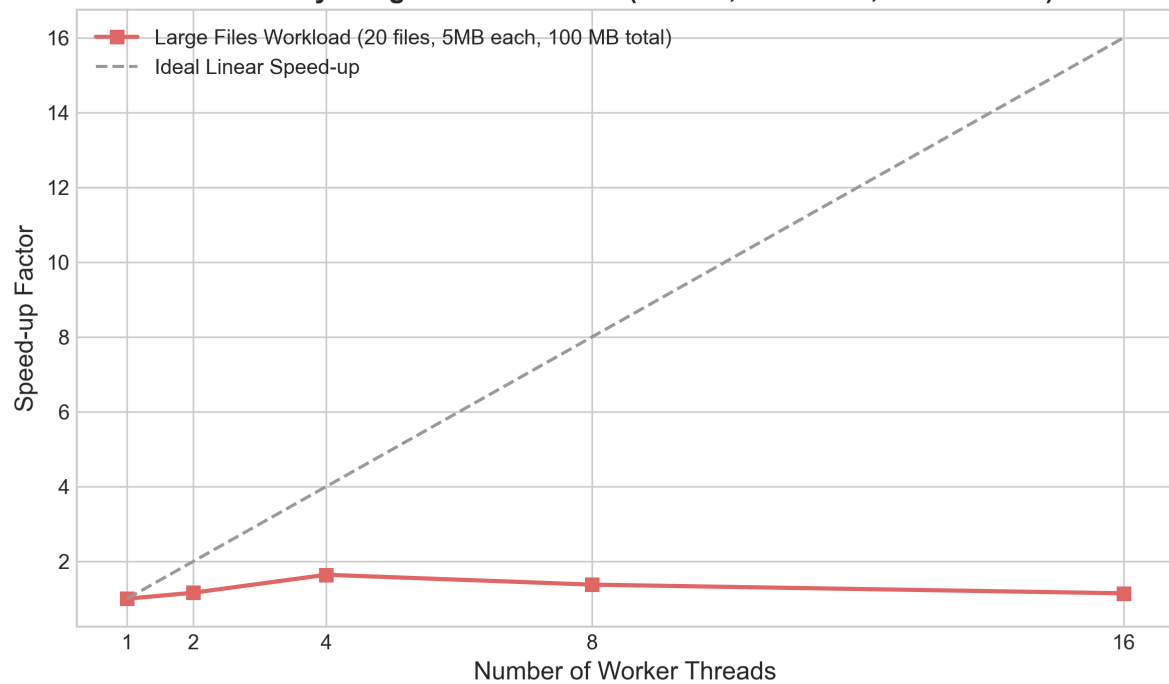
Curve di Speedup

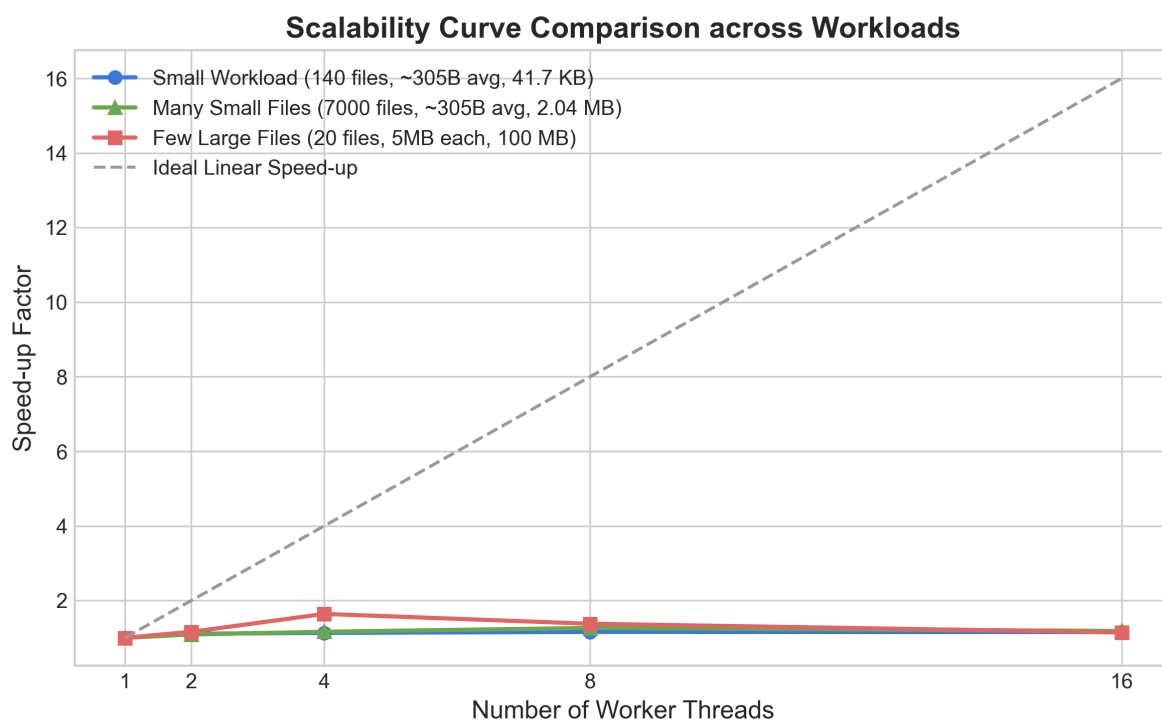


Scalability: Large Workload (7000 files, ~305B avg, 2.04 MB total)



Scalability: Large Files Workload (20 files, 5MB each, 100 MB total)



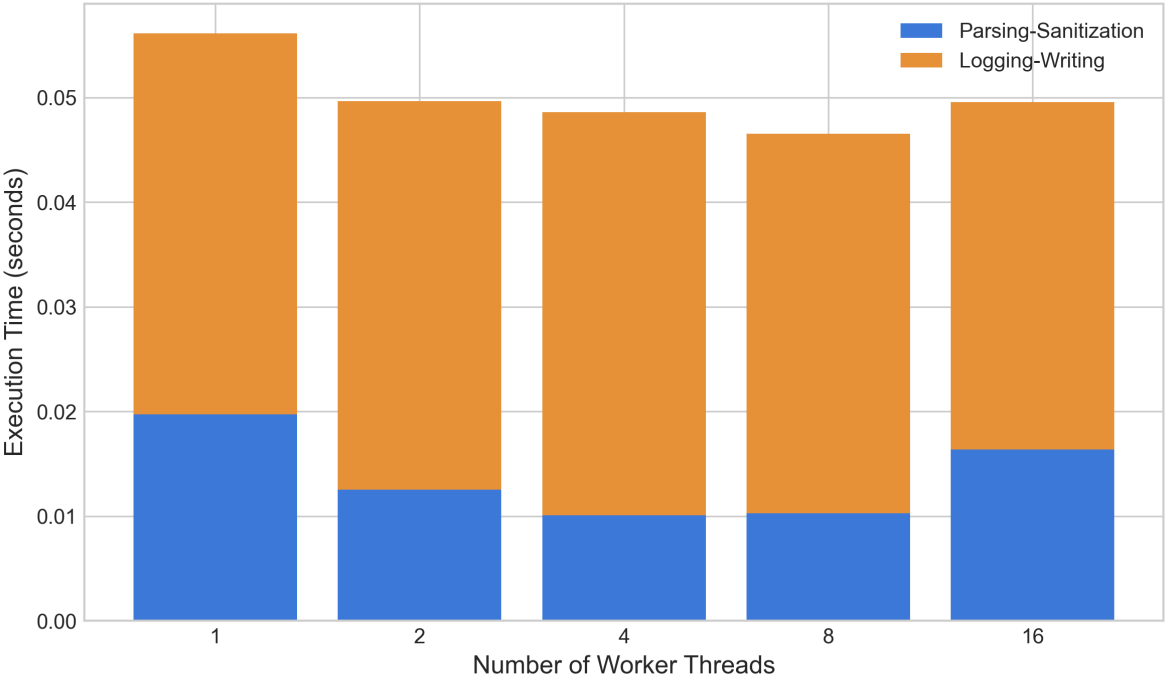


Suddivisione dei Tempi delle Fasi (Parsing vs Scrittura/Logging)

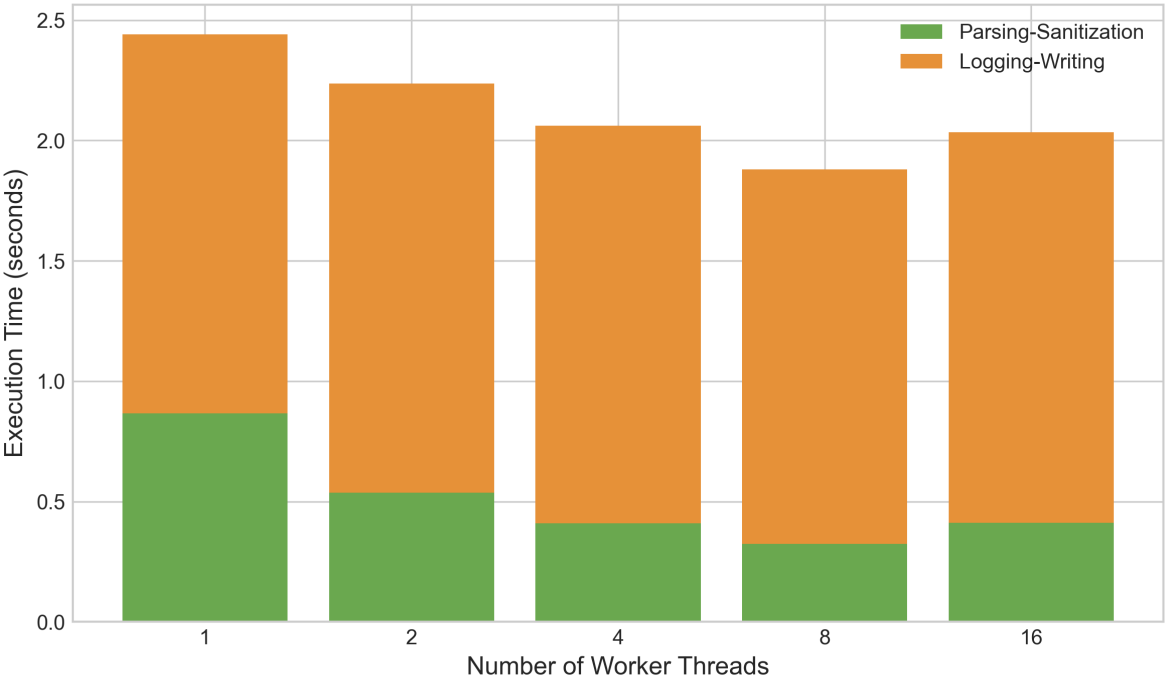
Thread	Parse Piccolo	Scrittura Piccolo	Parse Grande	Scrittura Grande	Parse File Grandi	Scrittura File Grandi
1	0.020 s	0.036 s	0.865 s	1.577 s	3.718 s	0.019 s
2	0.013 s	0.037 s	0.537 s	1.699 s	3.190 s	0.027 s
4	0.010 s	0.039 s	0.409 s	1.652 s	2.256 s	0.020 s
8	0.010 s	0.036 s	0.323 s	1.557 s	2.703 s	0.010 s
16	0.016 s	0.033 s	0.411 s	1.623 s	3.252 s	0.010 s

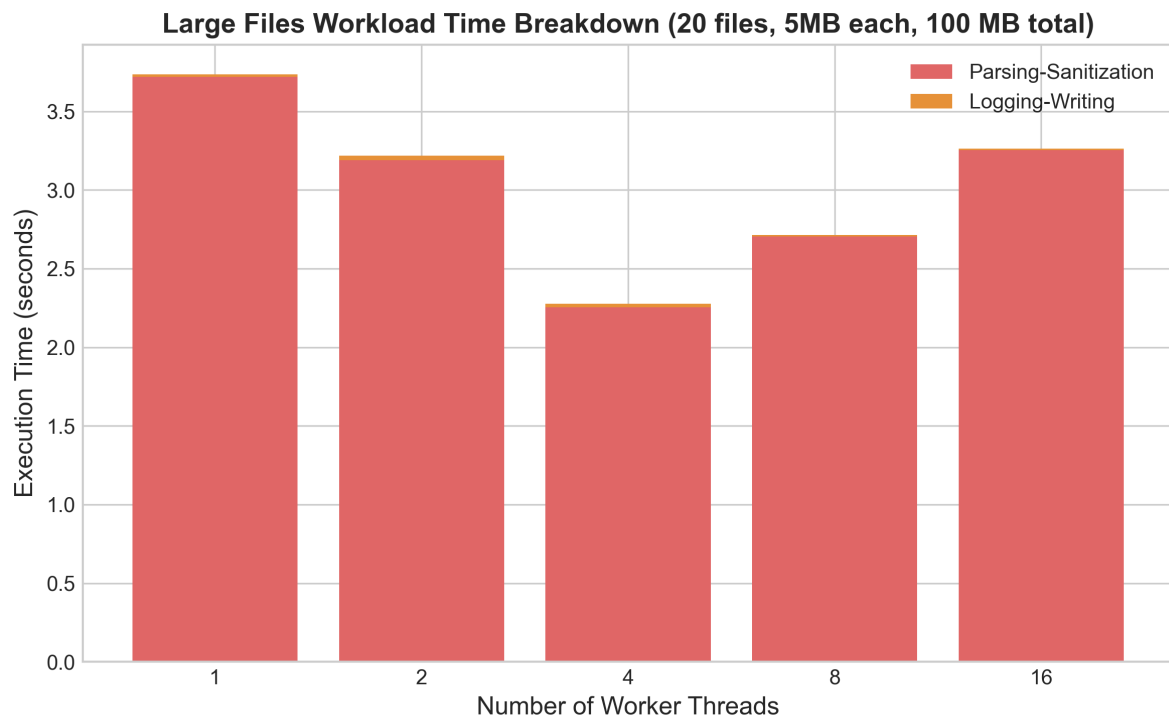
Suddivisione dei Tempi di Elaborazione

Small Workload Time Breakdown (140 files, ~305B avg, 41.7 KB total)



Large Workload Time Breakdown (7000 files, ~305B avg, 2.04 MB total)





Osservazioni e Confronto Critico sui Tre Carichi:

1. Analisi del Carico "Pochi File Grandi" (20x5MB):

- **Impatto Trascurabile del Logging:** A differenza del carico da 7000 file (dove la serializzazione JSON di 7000 elementi impiega un tempo fisso di ~1,6s), per il carico con 20 file grandi il tempo di scrittura finale del report è quasi istantaneo (**~0.01s - 0.02s**).
- **Dominanza del Parsing CPU-bound:** L'intero tempo di esecuzione è concentrato nella fase di parsing ed elaborazione HTML (**3.718s** con 1 thread). Aumentando i thread a 4, la durata di parsing scende a **2.256s**, registrando uno speedup di **1.64x**.
- **Contesa Hardware sui Core:** Superati i 4 thread worker (su architetture con core ad alte prestazioni/efficienza), la contesa delle risorse di calcolo CPU sul singolo payload da 5MB e la concorrenza sull'allocatore di memoria riducono l'efficienza aggiuntiva di ulteriori thread.

2. Confronto tra i Tre Criteri:

- **Pochi File Piccoli:** Il tempo totale (~54ms) è dominato dall'overhead fisso di inizializzazione del runtime e dalla scrittura dei file. Aumentare i thread offre un beneficio minimo (~1.16x).
- **Tanti File Piccoli:** Il tempo di parsing beneficia della scalabilità parallela (da 0,865s a 0,323s, speedup ~2.68x sul solo parser), ma lo speedup totale è limitato a ~1.27x a causa del tempo costante di serializzazione JSON finale (~1,6s) per l'array di 7000 report.
- **Pochi File Grandi:** Evita il collo di bottiglia della serializzazione JSON (solo 20 voci nel report) e permette alla componente CPU-bound del parser di mostrare la propria efficienza computazionale fino a 4-8 thread.

4.4 Consumo di Risorse

Durante l'intera suite di valutazione sperimentale, l'occupazione massima della RAM è stata pari a:

Picco Resident Set Size (RSS): 51,91 MB

Analisi dell'Efficienza Zero-Copy:

Il ridottissimo consumo di memoria è conseguenza diretta dell'approccio zero-copy:

1. **Token in Flusso:** `lol_html` evita di allocare un DOM in memoria, garantendo un footprint costante.
 2. **Condivisione tramite Riferimenti:** Elementi pesanti (come i dizionari di domini processati e la configurazione del sanitizer) sono condivisi tra i thread tramite riferimenti atomici `Arc<T>` senza alcuna duplicazione di dati.
-

5. Valutazione Critica

5.1 Limitazioni del Sistema

1. **Analisi Statica Limitata:**
Cassandra si basa esclusivamente su controlli statici delle stringhe e blacklist. Non è in grado di rilevare minacce generate dinamicamente a runtime tramite offuscamenti Javascript complessi (es. uso di `eval` dinamici o concatenazione di stringhe).
2. **Collo di Bottiglia sul Logging Sequenziale:**
La serializzazione del report JSON finale tramite `serde_json::to_writer_pretty` viene eseguita in modalità sincrona e monothread, rappresentando un evidente limite prestazionale sui sistemi con molti core ed elevata concorrenza.
3. **Limiti di Memoria sulle Risorse Scaricate:**
Sebbene `total_bytes` prevenga download infiniti complessivi, i singoli asset remoti vengono completamente scaricati in memoria prima di essere sniffati e sanitizzati. In caso di molteplici download concorrenti ad alta velocità, la memoria heap potrebbe subire una temporanea saturazione.

5.2 Possibili Estensioni Future

1. **Scrittura Asincrona del JSON in Streaming:**
Sostituzione dell'attuale serializzazione finale sincrona con un serializzatore JSON asincrono e in flusso (es. scrittura incrementale degli eventi man mano che avvengono) per rimuovere il collo di bottiglia del thread di log.
2. **Sandboxing Javascript via WebAssembly:**
Integrazione di un interprete Javascript super-leggero all'interno di una sandbox

WebAssembly (es. tramite `wasmtime`) per valutare gli script a runtime in sicurezza ed estrapolare minacce offuscate.

3. **Ricaricamento a Caldo delle Policy:**

Utilizzo di librerie di monitoraggio del filesystem (es. il crate `notify`) per aggiornare le definizioni delle regole TOML in tempo reale senza dover interrompere e riavviare Cassandra.

4. **Migliore posizionamento nei file:**

Gli errori che fanno riferimento al contenuto dei file usano un range di offset. Usare coppie righe/colonne sarebbe migliore, ma `lol_html` supporta solo offset (<https://github.com/cloudflare/lol-html/issues/157>).

5. **Streamed parsing per altre risorse:**

Solo i file html vengono letti in streaming, le altre risorse vengono prima lette completamente e poi sanitizzate.

Non ci sono librerie simili a `lol_html` per altri linguaggi, ma si potrebbe utilizzare `winnow::stream` .

6. **Supporto per altri tipi di risorse:**

Per esempio, il programma non supporta i file `svg` .